

LCAPIX

LIFE CYCLE ASSESSMENT PLATFORM

Onboarding Pack

Understand the platform, run the full workflow, and go
from a base case to a comparative analysis.

Prepared for onboarding · <https://lcapix.vercel.app>

Part 1 — Onboarding Manual

LCAPIX — Onboarding Manual

Welcome to **LCAPIX**, a web-based **Life Cycle Assessment (LCA)** platform for measuring and comparing the environmental impact of manufacturing processes. This manual is your single starting point: it explains what the platform is, the concepts behind it, how to log in, and how to run the complete workflow end-to-end — then points you to the deeper reference docs.

New here? Read sections 1–3 to understand the platform, then follow section 5 (the hands-on walkthrough) with the app open in another tab. Budget ~30 minutes.

1. What is LCAPIX?

LCAPIX helps manufacturers answer one question: **"What is the environmental footprint of making this product, and how do I reduce it?"**

You model a production process as a tree, record the resources it consumes and emits, and the platform calculates its impact across standard environmental categories (carbon, acidification, resource depletion, etc.) following recognized LCA methodologies (CML 2001, ReCiPe, TRACI). You can then create *alternative* versions of the process and compare them side by side to see which is greener.

Who uses it: sustainability analysts, process engineers, and product teams who need defensible environmental numbers for a product or process.

What it replaces: spreadsheets and desktop-only LCA tools — LCAPIX is web-based, multi-user, and collaborative.

2. Core concepts (the mental model)

Concept	What it is
Project	The top container for a body of LCA work (e.g. "Coffee Mug LCA"). Holds cases.
Case	One scenario within a project. Either a Base case (your reference/as-is process) or a Comparative case (an alternative you want to test against the base).
Component	A node in the process tree. Components form a 5-tier hierarchy (see below).
Flow	A substance going in (input) or out (output) of a component — e.g. "2.5 kWh electricity in". Flows are the raw data the math runs on.
Substance	An entry from the shared library (Electricity, Natural Gas, Aluminum, ...) that a flow references.
Impact category	An environmental metric the assessment computes — Global Warming (kg CO ₂ eq), Acidification (kg SO ₂ eq), etc.
Assessment (run)	A calculation over a case that turns its flows into impact numbers using a chosen methodology.
Comparison	A side-by-side diff of two or more cases' assessment results.

The 5-tier process hierarchy

Every process is modeled top-down as a tree. Each level is a child of the one above:

- | | | |
|---------------------|--------------------------|-----------------------------------|
| 1. PRODUCT / SYSTEM | the thing being assessed | e.g. "Ceramic Coffee Mug (350ml)" |
| └ 2. MACHINE / LINE | major production stage | e.g. "Forming Line" |
| └└ 3. SUBPROCESS | a step within that stage | e.g. "Slip Casting" |

- └ 4. OPERATION a specific operation e.g. "Mold Filling"
- └ 5. ELEMENTAL TASK where flows attach e.g. "Mold Cleaning"

Flows (energy, materials, emissions) attach to the lower nodes. Only nodes with flows contribute to the results — so a tree with no flows produces all-zero impacts.

3. Access & environment

Production app	https://lcapix.vercel.app
Sign up	https://lcapix.vercel.app/auth/signup
Log in	https://lcapix.vercel.app/auth/login
Hosting	Vercel (Next.js app)
Database	AWS RDS MySQL (<code>lca_v3</code>) — where all projects, cases, and results are stored

Create your own account from the signup page, or ask the team for a shared demo login.

4. What the platform looks like (the main screens)

- **/home** — your dashboard: all projects, plus + **New Project**.
- **Project page** — lists the project's cases; actions: + **Add Case**, **Compare Cases**.
- **Case editor** — the visual process tree (drag-and-drop). Add components, click a node to open the inspector and add flows, then **Run Assessment**.
- **Results page** — impact categories, top contributors, per-flow breakdown, a history timeline, and the ✨ **Magic Insights** panel (plain-language summary of the results and how to cut impact).
- **Analytics page** — side-by-side comparison charts and per-component deltas between cases.

5. The complete workflow (hands-on)

This is the core loop. Full click-by-click + API version is in **GETTING_STARTED.md**; a real worked example (a ceramic coffee mug, with actual numbers) is in **SAMPLE_CASE.md**.

1. **Create a project** — `/home` → + **New Project** → name + description.
2. **Add a base case** — project page → + **Add Case** → type **Base**.
3. **Build the hierarchy** — in the case editor, add a Product, then a Machine/Line under it, then Subprocess → Operation → Elemental Task. Each is a child of the one above.
4. **Add flows** — click the lower nodes → + **Add Flow** → pick a substance, set direction (input/output), quantity, and unit. This is the inventory.
5. **Run the assessment** — **Run Assessment** → pick methodology **CML 2001** → confirm. Results appear across the impact categories.
6. **Review results** — open **View Results**; explore the charts and **Magic Insights**.
7. **Add a comparative case** — project page → + **Add Case** → type **Comparative**. Rebuild the tree with your change (e.g. a recycled material or less energy) and run its assessment.
8. **Compare** — project page → **Compare Cases** → pick the base and the comparative case → read the side-by-side deltas on the analytics page.

Worked result (from the sample): a ceramic mug's Global Warming impact dropped from **6.09** → **3.75 kg CO₂ eq** (≈ **–38%**) when switching to a 50% recycled-clay variant — exactly the kind of decision the compare view is built to support.

6. Documentation index (what to read next)

Everything lives in `lca project v3/docs/`. Suggested reading order for a new team member:

Order	Document	What it covers
1	ONBOARDING_MANUAL.md (this file)	Platform overview, concepts, the big picture
2	GETTING_STARTED.md	From-zero walkthrough: project → base → comparative → analysis, UI + API
3	SAMPLE_CASE.md	A fully worked example with real IDs and numbers
4	CUSTOMER_ZERO_TO_ONE.md	The end-user's first-run journey / narrative
5	PUBLIC_API_GUIDE.md	REST API + the external data integrations (BLS, EIA, metals, PubChem, openLCA)
6	E2E_TEST_GUIDE.md	Step-by-step test walkthrough of every feature
7	DB_MIGRATION.md	Database structure + how the data is stored/moved (ops)

Background / methodology (one level up, in the `lca/` folder): - `Manual(a).pdf` and `LCAPIX_MANUAL_*.docx` — the original product manual. - `Documentation_*_Schema_*.md` — the data schema (v1 → v3), field definitions, and sample data. - `Sustainable Deck.pdf` — the product/vision deck. - `V0_Complete_User_Journey_Guide.md` — detailed hierarchy rules and edge cases per tier.

7. Glossary

- **LCA (Life Cycle Assessment)** — a standardized method (ISO 14040/14044) for quantifying the environmental impact of a product or process across its life cycle.
 - **Functional unit** — the reference basis for the assessment (e.g. "1 mug", "1 kWh").
 - **Inventory (LCI)** — the full set of input/output flows for the process.
 - **Characterization / impact category** — converting inventory flows into an environmental metric (e.g. kg of various gases → kg CO₂ equivalent for Global Warming).
 - **Methodology** — the factor set used for characterization (CML 2001, ReCiPe, TRACI).
 - **Base vs Comparative case** — reference scenario vs an alternative measured against it.
-

8. Getting help

- Start with the doc index in section 6 — most questions are answered there.
- For API/integration specifics, see `PUBLIC_API_GUIDE.md`.
- For anything unclear, reach out to the LCAPIX team.

Welcome aboard — the fastest way to "get it" is to create a throwaway project and run the section-5 loop once end to end.

Part 2 — Getting Started (From Zero to a Comparative Analysis)

Getting Started — From Zero to a Comparative Analysis

A clean, from-scratch walkthrough: **create an account** → **new project** → **base case** → **build the hierarchy** → **run an analysis** → **add a comparative case** → **compare them**.

Both the **UI clicks** and the equivalent **API calls** are shown for every step. For a fully worked example with real IDs and numbers (a ceramic coffee mug), see [SAMPLE_CASE.md](#).

- **App:** <https://lcapix.vercel.app>
- **API base:** <https://lcapix.vercel.app/api>
- **Methodology used below:** CML 2001

All write endpoints use snake_case JSON bodies and (except signup/login) require an `Authorization: Bearer <token>` header.

Step 0 — Create an account (or log in)

UI: go to `/auth/signup`, enter name + email + password, submit → you land on `/home`. (Returning users use `/auth/login`.)

API:

```
BASE="https://lcapix.vercel.app"

# Sign up (first time)
curl -s -X POST $BASE/api/auth/signup \
  -H "Content-Type: application/json" \
  -d '{"username":"yourname","email":"you@example.com","password":"YourPassw0rd!"}'

# Log in and capture the JWT for every later call
TOKEN=$(curl -s -X POST $BASE/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"you@example.com","password":"YourPassw0rd!"}' \
  | python3 -c "import json,sys; print(json.load(sys.stdin)['token'])")
echo "$TOKEN" # a JWT starting with eyJ
```

Step 1 — Create a project

UI: on `/home`, click **+ New Project** → give it a name and description → submit.

API:

```
PROJ_ID=$(curl -s -X POST $BASE/api/projects \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"project_name":"My First LCA","description":"Getting-started project"}' \
  | python3 -c "import json,sys; print(json.load(sys.stdin)['project']['project_id'])")
echo "Project: $PROJ_ID"
```

Note the field is `project_name` (snake_case), not `name`.

Step 2 — Create the base case

The **base case** is your reference scenario — the "as-is" process you'll measure against.

UI: on the project page, click **+ Add Case** → name it (e.g. `Standard Production`), choose type **Base**, submit.

API:

```
CASE_ID=$(curl -s -X POST $BASE/api/projects/$PROJ_ID/cases \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"case_name":"Standard Production","case_type":"base","description":"Baseline scenario"}' \
| python3 -c "import json,sys; print(json.load(sys.stdin)['case']['case_id'])")
echo "Base case: $CASE_ID"
```

Step 3 — Build the process hierarchy

Model your process as a 5-tier tree, each node a child of the one above:

Product → **Machine/Line** → **Subprocess** → **Operation** → **Elemental Task**

UI: open the case editor → **+ Add Component** for each tier → drag each under its parent (or pick a parent from the dropdown).

API (`component_type` ∈ `product` , `machine_line` , `subprocess` , `operation` , `elemental_task`):

```
mk(){ curl -s -X POST $BASE/api/cases/$CASE_ID/components \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d "$1" \
| python3 -c "import json,sys; print(json.load(sys.stdin)['component']['component_id']); }

P=$(mk '{"component_name":"Product","component_type":"product"}')
M=$(mk '{"component_name":"Line","component_type":"machine_line","parent_component_id":$P}')
S=$(mk '{"component_name":"Subprocess","component_type":"subprocess","parent_component_id":$M}')
O=$(mk '{"component_name":"Operation","component_type":"operation","parent_component_id":$S}')
T=$(mk '{"component_name":"Task","component_type":"elemental_task","parent_component_id":$O}')
echo "product=$P line=$M sub=$S op=$O task=$T"
```

Step 4 — Add flows (the inventory)

A **flow** is a substance consumed (`input`) or produced (`output`) at a node. Flows are what the assessment math actually runs on — **a case with no flows produces all-zero results.**

UI: click a node → **+ Add Flow** in the inspector → pick a substance, direction, quantity, unit.

API:

```
# Browse available substances (note the id you want; e.g. Electricity=7, Natural Gas=10)
curl -s "$BASE/api/substances?limit=50" -H "Authorization: Bearer $TOKEN"

# Attach flows to leaf nodes
curl -s -X POST $BASE/api/components/$T/flows \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"substance_id":7,"flow_type":"input","quantity":2.5,"unit":"kWh"}'

curl -s -X POST $BASE/api/components/$O/flows \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"substance_id":10,"flow_type":"input","quantity":1.8,"unit":"m3"}'
```

Step 5 — Run the analysis

UI: in the case editor click **Run Assessment** → pick methodology **CML 2001** → (optional) region → confirm.

API:

```
RESP=$(curl -s -X POST $BASE/api/cases/$CASE_ID/assessments \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"calculation_method":"CML 2001","region_code":"NC"}')
RUN_ID=$(echo "$RESP" | python3 -c "import json,sys; print(json.load(sys.stdin)['assessment']['run_id'])")
echo "$RESP" | python3 -m json.tool # summary + total_impacts + per-component results
```

Results come back across impact categories such as **Global Warming (kg CO₂ eq)**, **Acidification (kg SO₂ eq)**, **Photochemical Oxidation (kg C₂H₄ eq)**, and **Resource Depletion (kg Sb eq)**.

View them — UI: **View Results** (or the ✨ Magic Insights badge). API:

```
curl -s $BASE/api/assessments/$RUN_ID -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

Step 6 — Add a comparative case

A **comparative case** is an alternative scenario (a design change, a greener input, a different process) measured against the base.

UI: project page → + **Add Case** → name it (e.g. `Greener Variant`), type **Comparative**, submit. The editor opens showing the base tree as a reference until you diverge.

API:

```
COMP_ID=$(curl -s -X POST $BASE/api/projects/$PROJ_ID/cases \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"case_name":"Greener Variant","case_type":"comparative","description":"Alternative scenario"}' \
| python3 -c "import json,sys; print(json.load(sys.stdin)['case']['case_id'])")
echo "Comparative case: $COMP_ID"
```

Now **repeat Steps 3–5 against \$COMP_ID** — rebuild the hierarchy and add flows with the changed quantities (e.g. less energy, a recycled material), then run its assessment. That gives the comparative case its own results to diff against the base.

Step 7 — Compare the two cases

UI: project page → **Compare Cases** → pick the **base** and the **comparative** case → the analytics page shows side-by-side impact charts and per-component deltas.

API:

```
curl -s -X POST $BASE/api/comparisons \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"comparison_name":"Base vs Greener","\case_ids":["$CASE_ID,$COMP_ID","\project_id":$PROJ_ID}'"
# → { success: true, message: "Cases verified. View comparison in analytics." }
```

Then open `/project/<PROJ_ID>/analytics` to read the comparison.

Quick reference

Action	Method & path
Sign up	POST <code>/api/auth/signup</code>
Log in	POST <code>/api/auth/login</code>
Create project	POST <code>/api/projects</code>
Add case (base/comparative)	POST <code>/api/projects/:projectId/cases</code>

Action	Method & path
Add component	POST /api/cases/:caseId/components
List substances	GET /api/substances
Add flow	POST /api/components/:componentId/flows
Run assessment	POST /api/cases/:caseId/assessments
Get results	GET /api/assessments/:runId
Compare cases	POST /api/comparisons

Rules of thumb - Bodies are snake_case (project_name , case_name , component_name , parent_component_id). - Build the hierarchy top-down — a child needs its parent's component_id first. - Add at least one flow before running an assessment, or every impact comes back zero. - case_type is base or comparative ; you compare 2–10 cases at once.

Where this data is stored

Everything you create is persisted to the app's **AWS RDS MySQL** database (lca_v3). See [SAMPLE_CASE.md](#) and the migration notes for details.

Part 3 — Sample Case (Worked Example)

Sample Case — Coffee Mug LCA (end-to-end walkthrough)

A complete worked example that takes you from a **new project** → **base case** → **5-tier hierarchy** → **flows** → **assessment** → **comparative case** → **case comparison**, with both the UI clicks and the equivalent API calls.

Everything below was executed live against production on **2026-07-07** using the shared test account, so the IDs and numbers are real — you can open the links and see them.

- **App:** <https://lcapix.vercel.app> (alias of `lca-project-v3.vercel.app`)
- **Test login:** `lcapix50@gmail.com` / `Lcapix@guerry123`
- **Methodology used:** CML 2001 · region NC

What we built

Thing	Name	ID	Link
Project	Sample: Coffee Mug LCA	38	/project/38
Base case	Standard Production	153	/project/38/case/153
Base assessment	Run #111	111	results
Comparative case	Recycled Clay Variant	154	/project/38/case/154
Comparative assessment	Run #112	112	—
Comparison	Standard vs Recycled Clay	—	analytics

Headline result: switching to 50% recycled clay (less kiln gas + less electricity) cuts the mug's cradle-to-gate footprint from **6.09** → **3.75 kg CO₂ eq**, a **~38% reduction**.

Step 0 — Log in and grab a token

```
BASE="https://lcapix.vercel.app"
TOKEN=$(curl -s -X POST $BASE/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"lcapix50@gmail.com","password":"Lcapix@guerry123"}' \
  | python3 -c "import json,sys; print(json.load(sys.stdin)['token'])")
echo "$TOKEN" # a JWT starting with eyJ
```

All write endpoints use snake_case bodies (`project_name` , `case_name` , `component_name` , ...) and require the `Authorization: Bearer $TOKEN` header.

Step 1 — Create a project

UI: `/home` → **+ New Project** → name it → submit.

```
curl -s -X POST $BASE/api/projects \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"project_name":"Sample: Coffee Mug LCA","description":"350ml ceramic coffee mug — sample walkthrough"}'
# → { success: true, project: { project_id: 38, ... } }
```

Step 2 — Create the base case

UI: project page → + **Add Case** → name `Standard Production`, type **Base**.

```
PROJ_ID=38
curl -s -X POST $BASE/api/projects/$PROJ_ID/cases \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"case_name":"Standard Production","case_type":"base","description":"Baseline ceramic process"}'
# → { success: true, case: { case_id: 153, ... } }
```

Step 3 — Build the 5-tier hierarchy

Each tier is a child of the one above: **Product** → **Machine/Line** → **Subprocess** → **Operation** → **Elemental Task**.

UI: open the case editor → + **Add Component** for each tier → drag under its parent (or pick a parent in the dropdown).

```
CASE_ID=153
mk(){ curl -s -X POST $BASE/api/cases/$CASE_ID/components \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d "$1" \
  | python3 -c "import json,sys; print(json.load(sys.stdin)['component']['component_id']);" }

P=$(mk '{"component_name":"Ceramic Coffee Mug (350ml)","component_type":"product"}')
M=$(mk '{"component_name":"Forming Line","component_type":"machine_line","parent_component_id":$P}')
S=$(mk '{"component_name":"Slip Casting","component_type":"subprocess","parent_component_id":$M}')
O=$(mk '{"component_name":"Mold Filling","component_type":"operation","parent_component_id":$S}')
T=$(mk '{"component_name":"Mold Cleaning","component_type":"elemental_task","parent_component_id":$O}')
echo "product=$P machine=$M subprocess=$S operation=$O task=$T"
# → product=233165 machine=233166 subprocess=233167 operation=233168 task=233169
```

Component types accepted by the API: `product`, `machine_line`, `subprocess`, `operation`, `elemental_task`.

Step 4 — Add flows (the inventory)

A **flow** is a substance consumed (`input`) or produced (`output`) at a node — the unit of LCA accounting. Look up `substance_id` first, then attach flows.

UI: click a node → + **Add Flow** in the inspector → pick substance, direction, quantity, unit.

```
# Find substance ids (Electricity=7, Natural Gas=10 on prod today)
curl -s "$BASE/api/substances?limit=50" -H "Authorization: Bearer $TOKEN"

# Electricity at the elemental task
curl -s -X POST $BASE/api/components/$T/flows \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"substance_id":7,"flow_type":"input","quantity":2.5,"unit":"kWh"}'

# Kiln firing gas at the operation
curl -s -X POST $BASE/api/components/$O/flows \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"substance_id":10,"flow_type":"input","quantity":1.8,"unit":"m3"}'
# → { success: true, flow: { flow_id: ..., is_driver: 1, ... } }
```

Step 5 — Run the assessment

UI: case editor → **Run Assessment** → pick **CML 2001** → confirm.

```
RESP=$(curl -s -X POST $BASE/api/cases/$CASE_ID/assessments \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"calculation_method":"CML 2001","region_code":"NC"}')
echo "$RESP" | python3 -m json.tool
RUN_ID=$(echo "$RESP" | python3 -c "import json,sys; print(json.load(sys.stdin)['assessment']['run_id'])")
```

Base case (Run #111) — total impacts:

Impact category	Value	Unit
Global Warming	6.09	kg CO ₂ eq
Acidification	0.0045	kg SO ₂ eq
Photochemical Oxidation	0.0639	kg C ₂ H ₄ eq
Resource Depletion	0.0000135	kg Sb eq

The `run_id` is nested at `assessment.run_id` (also mirrored at the top level as `run_id`).

Step 6 — View results

UI: **View Results** (or the pulsing ✨ Magic Insights badge) → </project/38/case/153/results>. Renders the impact categories, top contributors, per-flow breakdown, and a historical timeline.

```
curl -s $BASE/api/assessments/$RUN_ID -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
# keys: assessment, summary, results, total_impacts, component_breakdown
```

Magic Insights (✨ button on the results page) is fully client-side — it composes deterministic insight text (summary / "how to reduce 20%?" / cost-vs-CO₂ tradeoff / compare-to-base) from the assessment data. Safe to demo, no LLM call.

Step 7 — Create a comparative case

UI: project page → + **Add Case** → name `Recycled Clay Variant`, type **Comparative**. The editor opens with the base tree shown as a reference ("Synced from base · edit to diverge") until you add your own components.

```
curl -s -X POST $BASE/api/projects/$PROJ_ID/cases \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"case_name":"Recycled Clay Variant","case_type":"comparative","description":"50% recycled raw clay"}'
# → { success: true, case: { case_id: 154, ... } }
```

Then rebuild the hierarchy (same as Step 3, but against `CASE_ID=154`) and add **lower** flow quantities to model the recycled-clay savings — here electricity `2.5 → 1.6 kWh` and kiln gas `1.8 → 1.1 m3` — and run its assessment (Run #112).

Comparative case (Run #112) — total impacts:

Impact category	Value	Unit
Global Warming	3.75	kg CO ₂ eq
Acidification	0.0029	kg SO ₂ eq
Photochemical Oxidation	0.0391	kg C ₂ H ₄ eq

Impact category	Value	Unit
Resource Depletion	0.0000086	kg Sb eq

Step 8 — Compare the cases

UI: project page → **Compare Cases** → pick **Standard Production** (base) and **Recycled Clay Variant** (comp) → the analytics page shows side-by-side impact charts and per-component deltas.

```
curl -s -X POST $BASE/api/comparisons \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"comparison_name":"Standard vs Recycled Clay","case_ids":[153,154],"project_id":38}'
# → { success: true, message: "Cases verified. View comparison in analytics." }
```

Delta (Base → Recycled):

Impact category	Base	Recycled	Change
Global Warming	6.09	3.75	−38.4%
Acidification	0.0045	0.0029	−36.0%
Photochemical Oxidation	0.0639	0.0391	−38.8%
Resource Depletion	0.0000135	0.0000086	−36.0%

View: </project/38/analytics>

One-shot script

```
#!/usr/bin/env bash
set -euo pipefail
BASE="https://lcapix.vercel.app"
TOKEN=$(curl -s -X POST $BASE/api/auth/login -H "Content-Type: application/json" \
  -d '{"email":"lcapix50@gmail.com","password":"Lcapix@guerry123"}' \
  | python3 -c "import json,sys; print(json.load(sys.stdin)['token'])")
auth=(-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json")
jid(){ python3 -c "import json,sys;print(json.load(sys.stdin)$1)"; }

PROJ=$(curl -s -X POST $BASE/api/projects "${auth[@]}" \
  -d '{"project_name":"Sample: Coffee Mug LCA","description":"walkthrough"}' | jid '['project']['project_id']")
CASE=$(curl -s -X POST $BASE/api/projects/$PROJ/cases "${auth[@]}" \
  -d '{"case_name":"Standard Production","case_type":"base"}' | jid '['case']['case_id']")
mk(){ curl -s -X POST $BASE/api/cases/$CASE/components "${auth[@]}" -d "$1" | jid '['component']['component_id']; }
P=$(mk '{"component_name":"Ceramic Coffee Mug (350ml)","component_type":"product"}')
M=$(mk '{"component_name":"Forming Line","component_type":"machine_line","parent_component_id":$P}')
S=$(mk '{"component_name":"Slip Casting","component_type":"subprocess","parent_component_id":$M}')
O=$(mk '{"component_name":"Mold Filling","component_type":"operation","parent_component_id":$S}')
T=$(mk '{"component_name":"Mold Cleaning","component_type":"elemental_task","parent_component_id":$O}')
curl -s -X POST $BASE/api/components/$T/flows "${auth[@]}" -d '{"substance_id":7,"flow_type":"input","quantity":2.5,"unit":"kWh"}'
curl -s -X POST $BASE/api/components/$O/flows "${auth[@]}" -d '{"substance_id":10,"flow_type":"input","quantity":1.8,"unit":"m3"}'
curl -s -X POST $BASE/api/cases/$CASE/assessments "${auth[@]}" -d '{"calculation_method":"CML 2001","region_code":"NC"}' \
  | jid '['total_impacts']"
echo "Project $PROJ / base case $CASE created."
```

Notes & gotchas

- **Assessment scope:** only nodes that carry flows contribute. Here 2 of 5 components had flows (`Mold Filling` , `Mold Cleaning`), so `components_with_flows: 2` in the summary — that's expected, not an error.
- **Zero-flow cases still "succeed":** running an assessment on a case with no flows returns success with empty results. Add at least one input/output before you trust the numbers.
- **Integration field names** (Suggest-costs buttons) are strict: BLS/EIA want `state` , Metals wants the enum symbol (`STL` , not `steel`). See [PUBLIC_API_GUIDE.md](#) for the exact bodies.
- This walkthrough writes to the **shared production test account** — feel free to delete the `Sample: Coffee Mug LCA` project (ID 38) when you're done poking at it.